

Notes on Continuous Observation Spaces

1 Setting

Everything so far — Sarsa, n -step Sarsa, Q-learning, Monte Carlo control — assumes a *tabular* representation: $\hat{Q}(s, a)$ is looked up in a table indexed by a finite (and usually small) set of states $s \in \mathcal{S}$. Many environments of interest instead have a **continuous observation space** $\mathcal{S} \subseteq \mathbb{R}^d$: e.g. `MountainCar-v0` returns $(\text{position}, \text{velocity}) \in \mathbb{R}^2$, and `CartPole` returns a 4-vector of cart position, cart velocity, pole angle, and pole angular velocity. A table over $\mathbb{R}^d$ is not just large — it is uncountably infinite, so no tabular method can be applied directly, and two states that are numerically close (and should behave almost identically) would otherwise be treated as completely unrelated table entries.

The fix is to convert the continuous observation into a small set of discrete features and keep using ordinary tabular control (Sarsa, in the examples below) on top of that discretization. We cover two such feature maps: **state aggregation** (binning) and **tile coding** (overlapping, offset binnings).

2 State aggregation

State aggregation discretizes each dimension of the observation independently into b_i bins, then treats the resulting bin-index tuple as a discrete state. For d -dimensional observations with bins $b_1, \dots, b_d$, this partitions $\mathcal{S}$ into a grid of $\prod_i b_i$ cells.

2.1 Construction

Given per-dimension bounds $\text{low}, \text{high} \in \mathbb{R}^d$ (available from `env.observation_space`) and a bin count vector $\mathbf{b} = (b_1, \dots, b_d)$, define d sets of bin edges

$$B_i = \text{linspace}(\text{low}_i, \text{high}_i, b_i - 1), \quad i = 1, \dots, d,$$

i.e. $b_i - 1$ interior edges producing b_i bins along dimension i . A raw observation $o = (o_1, \dots, o_d)$ is mapped to the discrete state

$$s = (\text{digitize}(o_1, B_1), \dots, \text{digitize}(o_d, B_d)) \in \{0, \dots, b_1 - 1\} \times \dots \times \{0, \dots, b_d - 1\},$$

where $\text{digitize}(x, B)$ returns the index of the bin containing x . This is implemented as an `ObservationWrapper`: it leaves the environment's dynamics untouched and only rewrites the observation, so any tabular algorithm can be pointed at it unmodified, with $\hat{Q}$ now a table of shape $b_1 \times \dots \times b_d \times |\mathcal{A}|$ (e.g. $20 \times 20 \times 3$ for `MountainCar-v0` with $\mathbf{b} = (20, 20)$).

2.2 Sarsa on the aggregated environment

Control proceeds exactly as ordinary tabular Sarsa (see the Sarsa notes), just indexing $\hat{Q}$ by the bin tuple instead of a raw state:

Algorithm 1 Sarsa with state aggregation

```

1: Choose bins  $\mathbf{b}$ ; build the aggregation wrapper from low, high,  $\mathbf{b}$ 
2: Initialize  $\hat{Q}(s, a) \leftarrow 0$  for all bin-tuples  $s$  and actions  $a$ 
3: for episode = 1, 2, ... do
4:    $S_0 \leftarrow$  aggregated reset observation; choose  $A_0$   $\varepsilon$ -greedily from  $\hat{Q}(S_0, \cdot)$ 
5:   for  $t = 0, 1, 2, \dots$  until done do
6:     Take  $A_t$  in the wrapped env, observe  $R_{t+1}$  and aggregated  $S_{t+1}$ 
7:     Choose  $A_{t+1}$   $\varepsilon$ -greedily from  $\hat{Q}(S_{t+1}, \cdot)$ 
8:      $\hat{Q}(S_t, A_t) \leftarrow \hat{Q}(S_t, A_t) + \alpha(R_{t+1} + \gamma \hat{Q}(S_{t+1}, A_{t+1}) - \hat{Q}(S_t, A_t))$ 
9:   end for
10: end for
```

2.3 Limitations

Aggregation introduces a hard discontinuity at every bin edge: two observations o, o' on opposite sides of an edge but arbitrarily close in $\mathbb{R}^d$ can map to completely different, independently-learned table entries, while two observations far apart within the same bin are treated as identical. Resolution is also coupled across the whole space — finer bins improve accuracy everywhere but blow up the table size ($\prod_i b_i$ grows exponentially in d) and slow down learning (each cell must be visited many times before $\hat{Q}$ is accurate there).

3 Tile coding

Tile coding keeps the idea of binning but overlays n independent aggregations (**tilings**) of the same space, each shifted by a different random offset. A state is then represented not by one bin index but by n bin indices, one per tiling, and $Q(s, a)$ is estimated as the *average* of the n corresponding table entries. This is a coarse form of linear function approximation with binary features: the effective resolution is much finer than any single tiling's bin width, because two nearby points that fall in the same bin for most tilings but different bins for a few tilings get almost — but not exactly — the same estimate.

3.1 Construction

For n tilings and base bin counts $\mathbf{b}$, tiling $i = 1, \dots, n$ uses independently jittered bounds

$$\text{low}^{(i)} = \text{low} - u_i \cdot 0.2 \cdot \text{low}, \quad \text{high}^{(i)} = \text{high} + u_i \cdot 0.2 \cdot \text{high}, \quad u_i \sim \text{Uniform}(0, 1),$$

together with a per-tiling displacement (an asymmetric offset following the displacement vector $(1, 3, 5, \dots)$ standard in tile coding, so that no two tilings' bin edges coincide) before computing per-dimension edges $B_1^{(i)}, \dots, B_d^{(i)}$ exactly as in state aggregation. A raw observation o is then mapped to a list of n bin-tuples,

$$s = (s^{(1)}, \dots, s^{(n)}), \quad s^{(i)} = (\text{digitize}(o_1, B_1^{(i)}), \dots, \text{digitize}(o_d, B_d^{(i)})),$$

and $\hat{Q}$ is stored as an $n \times b_1 \times \dots \times b_d \times |\mathcal{A}|$ table — one full aggregation table per tiling.

3.2 Action selection and updates

The action-value at s is the mean over tilings,

$$\hat{Q}(s, a) = \frac{1}{n} \sum_{i=1}^n \hat{Q}^{(i)}(s^{(i)}, a),$$

used directly for ε -greedy action selection. Each of the n per-tiling tables is then updated independently with the same one-step Sarsa error, so that the tiling that happens to distinguish two nearby states most finely can still adapt on its own:

$$\hat{Q}^{(i)}(S_t^{(i)}, A_t) \leftarrow \hat{Q}^{(i)}(S_t^{(i)}, A_t) + \alpha \left(R_{t+1} + \gamma \hat{Q}^{(i)}(S_{t+1}, A_{t+1}) - \hat{Q}^{(i)}(S_t^{(i)}, A_t) \right), \quad i = 1, \dots, n.$$

Algorithm 2 Sarsa with tile coding

- 1: Choose n tilings and base bins $\mathbf{b}$; build n jittered/offset aggregations
 - 2: Initialize $\hat{Q}^{(i)}(s, a) \leftarrow 0$ for each tiling $i = 1, \dots, n$
 - 3: **for** episode = 1, 2, ... **do**
 - 4: $S_0 \leftarrow (S_0^{(1)}, \dots, S_0^{(n)}) \leftarrow$ tile-coded reset observation
 - 5: Choose A_0 ε -greedily from $\hat{Q}(S_0, \cdot) = \frac{1}{n} \sum_i \hat{Q}^{(i)}(S_0^{(i)}, \cdot)$
 - 6: **for** $t = 0, 1, 2, \dots$ until done **do**
 - 7: Take A_t , observe R_{t+1} and tile-coded S_{t+1}
 - 8: Choose A_{t+1} ε -greedily from $\hat{Q}(S_{t+1}, \cdot)$
 - 9: **for** $i = 1, \dots, n$ **do**
 - 10: $\hat{Q}^{(i)}(S_t^{(i)}, A_t) \leftarrow \hat{Q}^{(i)}(S_t^{(i)}, A_t) + \alpha (R_{t+1} + \gamma \hat{Q}^{(i)}(S_{t+1}, A_{t+1}) - \hat{Q}^{(i)}(S_t^{(i)}, A_t))$
 - 11: **end for**
 - 12: **end for**
 - 13: **end for**
-

4 State aggregation vs. tile coding

	State aggregation		Tile coding
Tables stored	1		n (one per tiling)
Effective resolution	bin width		finer than any single tiling's bin width
Generalization across nearby states	none (hard cell boundaries)		partial (shared tilings)
Memory	$\prod_i b_i \cdot \mathcal{A} $		$n \cdot \prod_i b_i \cdot \mathcal{A} $
Discontinuities at bin edges	all coincide, always visible		smoothed out (edges of different tilings rarely coincide)

Both are still fundamentally tabular under the hood — every table entry is updated independently by the same sample-average TD rule as ordinary Sarsa — so all of the tabular convergence guarantees still apply *to the discretized problem*. What is lost is any guarantee about the *original* continuous-state problem: a coarser discretization can introduce approximation error (aliasing states that should be distinguished) that no amount of additional training removes, and picking $\mathbf{b}$ and n is itself a bias–variance-style tradeoff between resolution/memory and how much data is needed to fill every cell.

4.1 Beyond binning

State aggregation and tile coding are the simplest members of a much larger family of **function approximation** methods for continuous (or just very large) state spaces, where $\hat{Q}(s, a; \theta)$ is represented parametrically (linear-in-features, or a neural network as in DQN) instead of as an explicit table, and the TD update becomes a gradient step on θ rather than a per-entry table update. Tile coding is in fact exactly linear function approximation with a particular fixed, binary feature map $\mathbf{x}(s) \in \{0, 1\}^{n \cdot \prod_i b_i}$ (one active feature per tiling); it is presented here in tabular form because the per-tiling tables can be updated independently with the ordinary tabular Sarsa rule, without needing to introduce gradients explicitly.

5 Reference

Sutton, R. S., & Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd ed., Section 9.5.4 (Tile Coding).